

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Simulation Task

COURSE NAME: Embedded Systems

COURSE CODE: ECA14

COURSE OUTCOME COVERED

CO2: Apply Embedded C programming concepts, including structured design, real-time constraint handling, and System C-based modeling, to develop and analyze embedded applications for industrial use cases. (BL5)

Assessment Tool Weightage:	30%
Assessment Tool	: Simulation Task
Duration	: Self Learning
Marks	: 30
Type of Assessment conduction	: Self Learning
Topics Covered	: Embedded C programming: Features of embedded programming languages, Difference between C and Embedded C, Key Characteristics of Embedded C, Standard Embedded C Data Types, Block Diagram Explanation of Embedded C, Basic Embedded C Programming Steps, Vibration Monitoring Node design, Industrial HVAC Control Firmware design

Description

- Assessment will be carried out through rubric-based evaluation, where students complete the assigned task independently.
- Simulation will be done in SystemC, for the given specification and interpret the results.
- Each question will be assigned to one student, and he/she is required to collaboratively discuss the given problem and present his/her responses with clear justification.
- It will be upload in Github

Evaluation Type	: Embedded C/SystemC Simulation Assessment
Result Generation	: Rubric-based evaluation

QUESTIONS: 30

Total Marks: 30

S. No.	Question Bank	BTL	Marks
1	Develop a detailed requirements specification for a smart home air quality monitoring system integrating sensing, processing, and user interaction.	BL3	30
2	Construct a simulation-based Embedded C application for a smart factory node with multi-sensor input, UART communication, and real-time analysis.	BL5	30
3	Develop an Embedded C simulation for an automotive engine monitoring system using ADC inputs and interrupt-driven sampling.	BL5	30
4	Design a simulated Embedded C firmware for a smart thermostat with modular design and timer-based control.	BL4	30
5	Construct an Embedded C simulation for a healthcare monitoring system with watchdog timers and fail-safe operation.	BL5	30
6	Develop a SystemC-assisted Embedded C simulation for an industrial robotic arm with concurrency and synchronization.	BL5	30
7	Create an Embedded C simulation for a vibration monitoring node using circular buffers and timer-based sampling.	BL4	30
8	Design an Embedded C simulation for an automotive ABS system with interrupt handling and real-time constraints.	BL5	30
9	Construct a simulated Embedded C program for a wearable healthcare device with serial communication and low power operation.	BL4	30
10	Develop an Embedded C simulation for a smart home lighting system using motion and ambient sensing.	BL3	30

11	Design an Embedded C simulation for an Industry 5.0 human-machine collaboration system with real-time interaction.	BL5	30
12	Construct a simulation-based Embedded C program for an industrial HVAC system with PID control.	BL5	30
13	Develop an Embedded C simulation for an automotive dashboard system handling multiple display updates.	BL3	30
14	Create a SystemC-integrated Embedded C simulation for a multi-sensor data acquisition system.	BL5	30
15	Design an Embedded C simulation for a healthcare infusion pump with timing and safety mechanisms.	BL5	30
16	Construct a simulated Embedded C program for a smart energy meter with data transmission.	BL4	30
17	Develop an Embedded C simulation for an automotive lane detection alert system.	BL5	30
18	Create a simulation-based Embedded C firmware for an industrial conveyor belt system.	BL4	30
19	Design an Embedded C simulation for a wearable fitness tracker with real-time data processing.	BL3	30
20	Construct a SystemC-based Embedded C simulation for a distributed industrial control system.	BL5	30
21	Develop a comprehensive Embedded C simulation integrating sensors, scheduling, and SystemC modeling.	BL5	30
22	Develop an Embedded C simulation for an automotive ECU with timer scheduling and watchdog recovery.	BL5	30
23	Construct a simulated Embedded C firmware for a smart HVAC controller with structured design.	BL4	30
24	Design an Embedded C simulation for a healthcare monitoring system with interrupt sampling and fail-safe mechanisms.	BL5	30
25	Develop a SystemC-integrated Embedded C simulation for an industrial robotic arm with concurrent task handling.	BL5	30

26	Create an Embedded C simulation for a vibration monitoring system using real-time processing.	BL4	30
27	Design an Embedded C simulation for an automotive ABS system under rapid braking conditions.	BL5	30
28	Construct an Embedded C simulation for a wearable healthcare device with low power and real-time operation.	BL4	30
29	Develop an Embedded C simulation for a smart home lighting system with sensing and control.	BL3	30
30	Create a comprehensive Embedded C simulation for an Industry 5.0 smart system with performance evaluation.	BL5	30

26. Create an Embedded C simulation for a vibration monitoring system using real-time processing.

AIM:

To implement a real-time vibration monitoring system using an Arduino Uno and SW-420 vibration sensor in Proteus using Embedded C.

ALGORITHM:

1. Start the system.
2. Initialize the sensor, LED, and buzzer pins.
3. Read the vibration sensor value.
4. If vibration is detected:
 - Turn ON the LED and buzzer.
5. Else:
 - Turn OFF the LED and buzzer.
6. Repeat the process continuously.
7. Stop.

SOURCE CODE:

```
#define SENSOR_PIN A0
#define LED_PIN 13
#define BUZZER_PIN 8

int threshold = 500;    // Adjust if needed

void setup()
{
    pinMode(LED_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);

    Serial.begin(9600);
}

void loop()
{
    int sensorValue = analogRead(SENSOR_PIN);

    Serial.print("Sensor Value: ");
    Serial.println(sensorValue);

    if(sensorValue > threshold)
    {
```

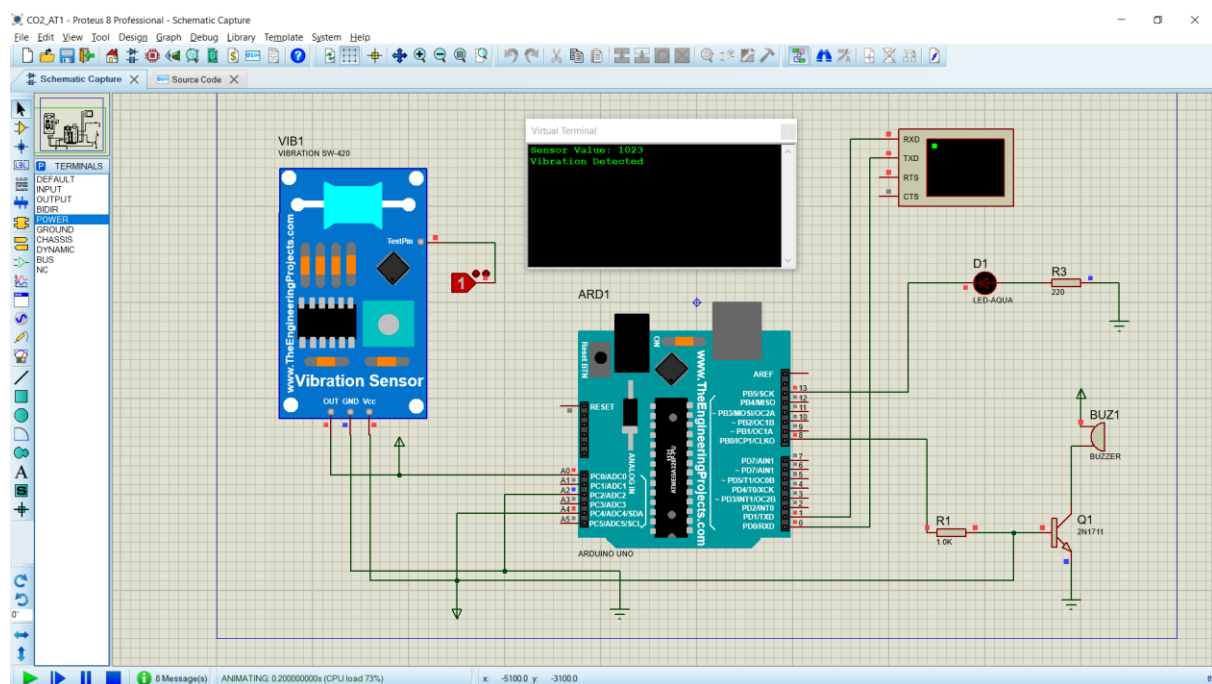
```

digitalWrite(LED_PIN, HIGH);
digitalWrite(BUZZER_PIN, HIGH);
Serial.println("Vibration Detected");
}
else
{
digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);
Serial.println("No Vibration");
}

delay(200);
}

```

OUTPUT:



RESULT:

The Embedded C simulation for the real-time vibration monitoring system was successfully implemented in Proteus. The system detected vibrations using the SW-420 vibration sensor and activated the LED and buzzer as alert indicators whenever vibration was detected.

Evaluation Rubric (30 Marks per Question)

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Requirements Analysis & Conceptual Understanding	20%	Clearly identifies all system requirements and demonstrates comprehensive understanding of the embedded application	Identifies most requirements with minor omissions	Partial understanding of requirements and concepts	Poor understanding of requirements and concepts
System Design & Simulation Model Development	25%	Develops a complete, modular, and technically accurate simulation/model independently	Develops a functional model with minor design issues	Develops a partially complete model requiring guidance	Unable to develop a proper simulation model
Implementation & Tool Usage	20%	Correctly implements the simulation using Embedded C/SystemC tools with proper execution and debugging	Implementation is mostly correct with minor errors	Limited implementation with several errors	Unable to implement or execute the simulation
Testing, Validation & Result Analysis	20%	Thoroughly tests the system, validates outputs, and provides accurate interpretation of results	Adequate testing and analysis with minor gaps	Limited testing and superficial analysis	Little or no testing and analysis performed
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

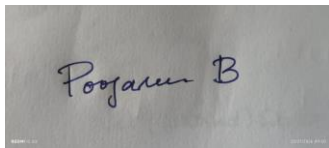
Performance Levels

Level	Description
Level 4 – Exemplary	Complete and technically accurate simulation with correct design, implementation, testing, and analysis.
Level 3 – Proficient	Functional simulation with minor technical errors and good analysis.
Level 2 – Developing	Partially functional simulation with noticeable design, implementation, or testing gaps.
Level 1 – Beginning	Incomplete simulation with limited understanding and execution.

Code of Conduct:

I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

A photograph of a piece of paper with the handwritten signature 'Poojasree B' in blue ink.

Signature of the student:

